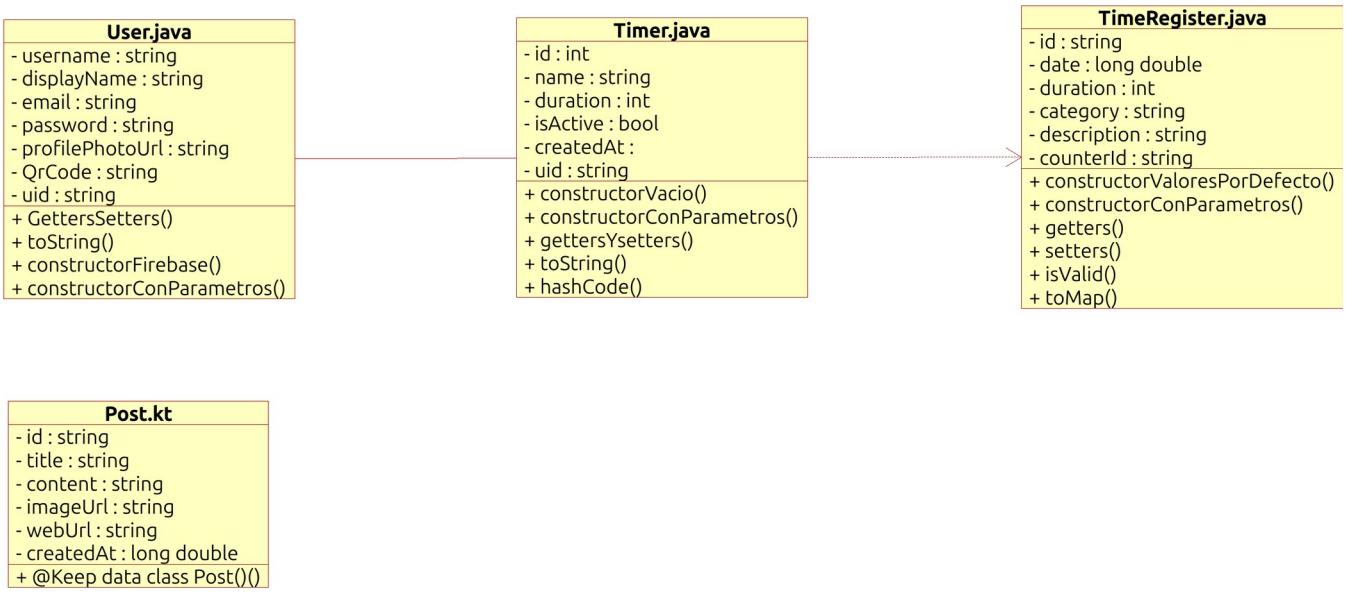
Diagrama de Clases

Temporis:

Una herramienta accesible para el control y la optimización de nuestro tiempo

Kevin Zamora Amela

Diagrama de Clases



1. Introducción

El presente documento contiene el **Diagrama de Clases UML** del proyecto **Temporis**. Este modelo representa la estructura estática del sistema, detallando los atributos, los métodos y las interrelaciones entre los objetos que componen la lógica de negocio de nuestra aplicación para Android.

2. Arquitectura de Clases y Encapsulamiento

El diseño sigue los principios de la Programación Orientada a Objetos (POO), haciendo especial énfasis en el **encapsulamiento**.

- **Atributos Privados (-):** La mayoría de los campos de datos se han definido como privados para proteger la integridad de la información, permitiendo su acceso únicamente a través de métodos públicos.

  A modo de ejemplo, se presentan los campos o parámetros privados, que se encuentran definidos en la clase User (o Usuario):

  ```
  private String username, displayName, email, password, profilePhotoUrl,
  QrCode, uid;
  ```

  ```
  private int rol;
  ```

- **Métodos Públicos (+):** Se detallan los constructores, *getters*, *setters* y métodos de utilidad (como `toMap()`), necesarios en algunas clases (como *Timer*), para la interacción con *Firebase Firestore*.

◆ Constructores:

```
public User() {
    this.username = "";
    this.displayName = "";
    this.email = "";
    this.password = "";
    this.profilePhotoUrl = "";
    this.QrCode = "";
    this.uid = "";
    this.rol = 0;
  }
```

```
public User(String uid, String username, String email,
  String displayName, String profilePhotoUrl) {
    this.uid = uid;
    this.username = username;
    this.email = email;
    this.displayName = displayName;
    this.profilePhotoUrl = profilePhotoUrl;
  }
```

◆ Métodos de encapsulado [para controlar el acceso] (Getters y setters):

```
public String getUid() { return uid; }
public void setUid(String uid) { this.uid = uid; }
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getDisplayName() { return displayName; }
public void setDisplayName(String displayName) { this.displayName = displayName; }
public String getProfilePhotoUrl() { return profilePhotoUrl; }
public void setProfilePhotoUrl(String profilePhotoUrl) { this.profilePhotoUrl =
profilePhotoUrl; }
public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }
public String getQrCode() { return QrCode; }
public void setQrCode(String qrCode) { this.QrCode = qrCode; }
public int getRol() { return rol; }
public void setRol(int rol) { this.rol = rol; }
```

- Método de impresión:

```
@Override
public String toString() {
  return "User{" +
      "username='" + username + '\'' +
      "displayName='" + displayName + '\'' +
      ", email='" + email + '\'' +
      ", password='" + password + '\'' +
      ", profilePhotoUrl='" + profilePhotoUrl + '\'' +
      ", QrCode='" + QrCode + '\'' +
      ", uid=" + uid +
      ", rol=" + rol +
      /*", decription=" + description +*/
      '}';
}
```

- Métodos auxiliares a destacar (Clases *Timer* y *TimeRegister*):

```
@Override
public boolean equals(Object o) {
  if (this == o) return true;
  if (!(o instanceof Timer)) return false;
  Timer timer = (Timer) o;
  return duration == timer.duration &&
      isActive == timer.isActive &&
      Objects.equals(id, timer.id) &&
      Objects.equals(name, timer.name) &&
      Objects.equals(createdAt, timer.createdAt) &&
      Objects.equals(uid, timer.uid);
}
```

```
@Override
public int hashCode() {
    return Objects.hash(id, name,
    duration, isActive, createdAt,
    uid);
}
```

```
public Map<String, Object> toMap() {
    Map<String, Object> map = new HashMap<>();
    map.put("id", id);
    map.put("date", date);
    map.put("duration", duration);
    map.put("category", category);
    map.put("description", description);
    if (counterId != null) { map.put("counterId", counterId); }
    return map;
}
```

3. Descripción de Relaciones UML

(Hacen referencia al diagrama presentado anteriormente)

Para este diagrama se han implementado las siguientes relaciones:

- **Asociación Simple:** Conecta la clase `User` con `Timer`. Se ha definido una cardinalidad de **1 a N (1..*)**, indicando que un usuario puede ser propietario de múltiples temporizadores, pero cada temporizador está vinculado a una única cuenta de usuario, mediante su identificador único (`uid`).
- **Asociación de Generación:** La clase `Timer` se relaciona con `TimeRegister`. Esta relación es clave para el módulo de estadísticas, ya que cada sesión de tiempo finalizada una instancia (de un objeto de "registro de tiempo"), y este resulta necesario para su persistencia de los datos en la nube.
- **Clase Independiente (Post):** La clase `Post` (desarrollada en Kotlin) se mantiene como una entidad de consulta para el "sección" de noticias/recursos. Dicha clase solamente realiza la lectura de datos cargados desde/alojados en Firebase Firestore. Y a su vez, esta demuestra la capacidad del proyecto, para manejar modelos de datos interoperables entre Java y Kotlin, ya que ambos lenguajes resultan altamente compatibles. PD: Kotlin viene a ser un "derivado" del lenguaje Java.

4. Justificación del Diseño

El diagrama refleja fielmente el código fuente alojado en el repositorio de Github (https://github.com/kevinzamoraa/Temporis-Final_Project-DAM/). Se ha priorizado una estructura limpia, mediante la metodología de Codigo Límpio o *Clean Code*, que facilite el mantenimiento futuro y la implementación de nuevas funcionalidades, como: la "Categorización avanzada" o los "Objetivos diarios", sin necesidad de reestructurar o modificar (en exceso) las "clases base".